

Занятие 7. Синтез: как разбирать новую архитектуру

Какой вопрос закрывает занятие

За шесть занятий мы разобрали довольно разные конструкции: full attention с KV-cache, finite-state sequence mixers, delta-family, SSM, гибриды, sparse attention, flow matching, Gaussian diffusion, score, masked diffusion и block diffusion. На уровне названий это легко превращается в каталог: одна paper предлагает Mamba, другая KDA, третья новый diffusion sampler, а через год появляется ещё одно название.

Такой каталог быстро устаревает.

Поэтому последнее занятие отвечает на другой вопрос:

как разобрать незнакомую архитектуру, если её названия в курсе вообще не было?

Представим, что в 2027 году выходит условная Mamba-4. Само слово Mamba почти ничего не должно решать за нас. Нужно открыть equations и последовательно выяснить:

- какой математический operator предложен;
- что хранится как state или cache;
- как выполняются read, write и forgetting;
- каким algorithm этот operator считается при training и decoding;
- какой generative objective и sampler используются, если это генеративная модель;
- какие координаты cost vector действительно уменьшились;
- что авторы доказали алгебраически или теоретически, а что только измерили в своей реализации.

Главный тезис занятия поэтому намеренно не звучит как «architecture X лучше architecture Y».

Курс даёт систему координат, в которой новую архитектуру можно разобрать до того, как принимать её название и benchmark table за объяснение.

Что нужно знать заранее

Новых тяжёлых математических конструкций здесь не появляется. Нужен уже пройденный материал.

Предполагается, что понятно:

- как работает full causal attention и зачем при autoregressive decode нужен KV-cache;
- чем persistent state отличается от workspace и HBM traffic;
- почему cost нельзя описать только FLOPs или big-O;
- как из causal linear sum получается состояние $S_t \in \mathbb{R}^{d_v \times d_k}$;
- чем parallel, recurrent и chunkwise forms отличаются как algorithms;
- как additive write, decay и delta rule по-разному изменяют finite state;
- зачем в Gated DeltaNet отдельно нужны forgetting и targeted edit;
- что SSD связывает structured state-space и matrix representations конкретных operators, а не доказывает равенство полных architectures;
- чем fixed state отличается от token-indexed KV-cache и sparse retrieval;
- что такое probability path, flow, Gaussian corruption и score;
- почему для текста нужен discrete bridge;
- чем masked diffusion отличается от обычного masked language modeling recipe;
- почему sampler steps, N_{FE} и число заново обработанных token positions являются разными величинами.

Сохраняется нотация курса. Индекс t обозначает position в sequence, а τ - непрерывное время generative process или noise level. Для state-based operators q_t, k_t обозначают vectors после feature map, а состояние ориентировано как $S_t \in \mathbb{R}^{d_v \times d_k}$.

Что должно получаться после чтения

После этого конспекта нужно уметь без новой лекции:

1. Получив незнакомый sequence layer, выписать его state или cache, read rule, write rule и forgetting rule.
2. Отдельно описать mathematical operator и algorithm, которым он вычисляется при training и decoding.
3. Для efficiency claim назвать координату cost vector, которую claim на самом деле касается.
4. Привести пример, когда одинаковый $M_{\text{persistent}}$ не означает одинаковый B_{HBM} .
5. Привести пример, когда одинаковый N_{FE} не означает одинаковый объем вычислений.
6. Отличить equality математических functions от equivalence evaluation algorithms и от similarity архитектурных принципов.
7. Из локального loss $\ell_t(S)$ вывести delta update и объяснить границы этой интерпретации.
8. Объяснить, почему наличие online-learning interpretation не определяет feature maps, convolutions, normalization, gates и остальные части architecture.

9. Для iterative generator отдельно назвать objective, probability/noise path, sampler, N_{FE} и число элементов, пересчитываемых за один NFE.
10. Читая paper, сначала отделить exact identity или theorem от reported empirical result.
11. Объяснить, почему measured speedup на одном GPU и в одной implementation не является свойством математической формулы.
12. По abstract, main equation и algorithm section новой статьи составить короткую техническую карточку, не опираясь на маркетинговое имя семейства.

Словарь занятия

Понятие	English term	Определение
Вектор стоимости	cost vector	Набор независимых характеристик вычисления: память, traffic, arithmetic, sequential depth, NFE и другие ресурсы.
Состояние	state	Данные, через которые прошлое воздействует на следующий computational step. В recurrent mixer state обычно имеет фиксированный shape.
Кэш	cache	Сохранённые результаты прошлых вычислений, которые можно корректно reuse вместо пересчёта. KV-cache causal Transformer - основной пример.
Правило чтения	read rule	Способ получить output из текущего state или cache.
Правило записи	write rule	Способ внести новую информацию в state или cache.
Забывание	forgetting / retention	Механизм, определяющий, какая часть уже записанной информации ослабляется или удаляется.
Mathematical operator	mathematical operator	Функция, связывающая входы с выходами независимо от конкретного алгоритма вычисления.
Evaluation algorithm	evaluation algorithm	Конкретный способ вычислить operator: parallel, recurrent, chunkwise или другой.
Эквивалентность операторов	operator equivalence	Равенство математических функций при одинаковых входах и одинаковой semantics.
Вычислительная эквивалентность	computational equivalence	Несколько algorithms точно вычисляют один operator, если игнорировать numerical roundoff.
Архитектурное сходство	architectural similarity	Наличие общего design principle у разных architectures без утверждения, что их operators совпадают.

Понятие	English term	Определение
Online-learning interpretation	online-learning interpretation	Представление recurrent update как одного или нескольких optimization steps над локальной memory objective.
Generative objective	generative objective	Training criterion, связанный с выбранной generative construction.
Sampler	sampler	Algorithm, превращающий обученную модель в процедуру получения sample.
Измеренный результат	empirical result	Величина, полученная experiment в конкретной реализации и при конкретных условиях.
Доказанный результат	proved result	Утверждение, следующее из явно сформулированных assumptions и математического аргумента.
Failure mode	failure mode	Ситуация, в которой механизм систематически теряет нужное свойство, качество или efficiency.

1. Возвращение к cost vector

Зачем эта тема здесь

Курс начался с утверждения, что «стоимость модели» - не одно число. После шести занятий видно, зачем это было нужно.

У full attention главная системная проблема длинного autoregressive context часто связана с растущим token-indexed KV-cache и его чтением. У recurrent state persistent context может не расти с длиной prefix, но появляются finite-state bottleneck, arithmetic обновления и последовательная зависимость. У diffusion или flow генератор может вообще не иметь аналогичного autoregressive KV-cache, зато один sample требует нескольких model evaluations.

Если всё это свернуть в один scalar вроде «память x число проходов», разные физические ресурсы окажутся перемножены просто потому, что нам хочется получить одно число.

Так делать дальше не будем.

Интуиция

Представим две модели.

Первая помещается в 20 GiB и делает один тяжёлый проход.

Вторая тоже помещается в 20 GiB, но делает двадцать более дешёвых проходов.

Из этих двух предложений невозможно определить, какая быстрее.

Нужно знать хотя бы:

- сколько из 20 GiB приходится на weights;
- сколько на persistent context;
- сколько на workspace;
- сколько bytes реально движется между HBM и вычислительными блоками;
- сколько arithmetic выполняется;
- сколько вычислений зависят друг от друга;
- что означает «проход»;
- сколько token positions обрабатывается в каждом таком проходе.

Получается не один порядок моделей «от дешёвой к дорогой», а многомерное пространство.

Понятия, которые вводятся

Новых компонентов нет. Возвращаем полный cost vector занятия 1:

$$\mathcal{C} = (M_{\text{weights}}, M_{\text{persistent}}, M_{\text{workspace}}, B_{\text{HBM}}, F, D_{\text{seq}}, N_{\text{FE}}, N_{\text{processed}}, B_{\text{max}}).$$

Здесь:

- M_{weights} - память под parameters;
- $M_{\text{persistent}}$ - данные, сохраняемые между последовательными generation steps;
- $M_{\text{workspace}}$ - временная рабочая память конкретного algorithm;
- B_{HBM} - memory traffic;
- F - арифметика;
- D_{seq} - sequential depth;

- N_{FE} - число фактических evaluations основной модели или обученного поля;
- $N_{\text{processed}}$ - число token positions, которые реально заново прошли через backbone;
- B_{max} - достижимая concurrency на данном hardware.

Важно слово **вектор**. Координаты связаны между собой инженерно, но ни одна из них не определяет остальные.

Формальное изложение

Почему одного scalar cost нет

Для autoregressive full attention KV-cache на уровне одного layer растёт с числом сохранённых positions. В упрощённой записи: $M_{\text{KV}} \propto T(d_k + d_v)$.

Но это утверждение только про persistent context storage.

На одном decode step новый query должен ещё прочитать relevant K/V. Для dense attention число interactions также растёт с длиной prefix.

Finite-state recurrence выглядит иначе. Для additive linear state: $S_t = S_{t-1} + v_t k_t^\top$, причём $S_t \in \mathbb{R}^{d_v \times d_k}$.

Размер persistent state относительно T постоянный: $M_{\text{persistent}} = O(d_k d_v)$.

Но это ничего не говорит о training algorithm. Простой recurrent loop имеет зависимость $S_{t-1} \rightarrow S_t \rightarrow S_{t+1}$, поэтому его sequential depth по sequence растёт с T . Chunkwise или scan-like algorithms меняют именно эту часть cost profile.

Для iterative generator появляется ещё одна независимая ось. Пусть evaluation j заново обрабатывает L_j token positions. Тогда

$$N_{\text{processed}} = \sum_{j=1}^{N_{\text{FE}}} L_j.$$

Одно число N_{FE} не сообщает значения L_j .

И даже $N_{\text{processed}}$ не даёт полный F : attention над 1024 positions и 1024 независимых дешёвых elementwise updates имеют совсем разную арифметику.

Пример

Контрпример 1. Одинаковый persistent cache - разный traffic

Пусть две implementations хранят один и тот же token-indexed KV pool размером 8 GiB.

Вариант А использует dense attention. Для очередного query основная attention branch обращается ко всему доступному pool.

Вариант В использует sparse retrieval и для дорогой main branch выбирает примерно одну шестнадцатую stored context. Сам KV pool при этом остаётся тем же: следующий query может выбрать другие blocks.

Получаем одинаковое $M_{\text{persistent}} = 8 \text{ GiB}$, но совершенно разный объём main-attention reads.

У варианта В при этом есть дополнительный index mechanism, поэтому нельзя автоматически утверждать, что total $B_{\text{НВМ}}$ уменьшился ровно в 16 раз. Но главное уже видно:

одинаковое количество хранимых bytes ничего не гарантирует о количестве прочитанных bytes.

Именно поэтому sparse attention и cache compression нельзя описывать одной координатой «память».

Контрпример 2. Одинаковый NFE - разное число обработанных positions

Пусть две text-generation systems имеют $N_{\text{FE}} = 16$.

Первая - full-sequence masked diffusion для output region длины 1024. Каждый NFE заново обрабатывает все 1024 positions: $N_{\text{processed}} = 16 \cdot 1024 = 16384$.

Вторая система на каждом NFE заново обрабатывает только current block из 64 positions, а completed prefix использует через cache: $N_{\text{processed}} = 16 \cdot 64 = 1024$.

NFE совпадает, а число заново обработанных token states отличается в 16 раз.

И это всё ещё не означает автоматически 16-кратную разницу latency: current block может читать длинный cached prefix, различаются attention shapes, kernel utilization и остальные компоненты.

Получается цепочка:

$$N_{\text{FE}} \nRightarrow N_{\text{processed}} \nRightarrow F \nRightarrow \text{latency}.$$

Стрелки здесь означают именно отсутствие однозначного вывода.

Можно построить и обратные примеры. Два algorithms могут иметь одинаковый F , но разный D_{seq} : один выполняет большой parallel matrix multiplication, другой - длинную цепочку recurrent updates.

Где здесь легко ошибиться

Ошибка 1. «Назовите complexity модели».

Сначала нужно спросить: complexity чего и в каком режиме?

Training, prefill и decode - разные задачи.

Ошибка 2. «У обеих моделей $O(T)$, значит они примерно одинаково быстрые».

Нет. Big-O не показывает constants, traffic, sequential depth, tensor shapes и качество kernels.

Ошибка 3. «У diffusion нет большого KV-cache, значит memory problem решена».

Нет. Уменьшился один вид persistent state. Workspace, weights, repeated full-sequence computations и HBM traffic остались.

Ошибка 4. «У recurrent model fixed state, значит training memory тоже $O(1)$ по T ».

Нет. Backward может требовать activations или recomputation. Mathematical state и training workspace - разные координаты.

Ошибка 5. «Главные оси стоимости - память и число проходов».

Это полезная историческая метафора, но как аналитическая модель она слишком грубая.

«Память» смешивает weights, persistent context и workspace. «Проход» смешивает NFE, sequential stages и количество элементов, обработанных внутри одного evaluation.

Что теперь понятно

Нельзя корректно спросить «какая architecture дешевле?» без режима работы и нужной координаты.

Для causal attention особенно важны размер token-indexed cache и traffic при его чтении.

Для recurrent state важны размер state, арифметика update и computational dependence между updates.

Для diffusion и flow важны стоимость одного model evaluation, N_{FE} , $N_{processed}$ и sequential structure sampler.

Исходная рамка «память × число проходов» на этом месте окончательно остаётся метафорой. Дальше используем полный cost vector.

2. Декомпозиция вместо списка названий

Ось А, контекст и backbone: что хранится, как читается, как записывается, что забывается.

Ось В, генеративная конструкция: как факторизуется распределение, какой путь, какой objective, какой сэмплер.

Карточка ниже сначала разбирает эти оси независимо и только затем смотрит, как они сочетаются в полной model.

Зачем эта тема здесь

Если cost vector отвечает на вопрос «что стоит вычисление», нужен второй инструмент для вопроса «что вообще делает architecture».

Самый ненадёжный способ - начать с названия family.

Например, слова linear attention , SSM , RNN , RWKV или diffusion полезны как навигационные ярлыки, но внутри каждого ярлыка может быть большой design space.

Поэтому сначала раскладываем механизм на независимые решения.

Интуиция

Для sequence mixer полезно представить memory system.

Тогда появляются простые вопросы:

- что физически хранится;
- как запрос получает информацию из хранилища;
- как новая информация туда попадает;

- что происходит со старой;
- можно ли обработать sequence parallel;
- что сохраняется между decode steps.

Эта декомпозиция намеренно немного скучная. В этом её достоинство: маркетинговое имя исчезает, а остаётся механизм.

Для генеративной модели поверх этого возникает второй слой вопросов:

- чему учится network;
- по какому probability path движется;
- какой algorithm потом строит sample.

Sequence architecture и generative procedure нельзя автоматически склеивать. Например, один и тот же Transformer-like backbone можно включить в разные generative constructions.

Понятия, которые вводятся

Для state-based sequence mixer используем семь вопросов.

1. Что является **state**?
2. Как выполняется **read**?
3. Как выполняется **write/update**?
4. Что и как **забывается**?
5. Какие **normalization и gates** добавлены?
6. Каким algorithm operator считается при **training**?
7. Каким algorithm operator считается при **decoding**?

Для generative model добавляются ещё четыре.

8. Какой **objective**?
9. Какой **probability path или noise path**?
10. Какой **sampler**?
11. Сколько **NFE** и сколько элементов обрабатывается на один NFE?

Для mechanism, который не является state-based mixer, некоторые клетки честно окажутся неприменимыми. Это нормальный результат анализа, а не недостаток checklist.

Формальное изложение

Full causal attention

Начнём с обычного causal attention.

Что хранится

При autoregressive decode persistent context - token-indexed KV-cache: $\{\hat{K}_i, V_i\}_{i=1}^t$.

Prefix представлен не одной compressed matrix фиксированного shape, а последовательностью записей.

Read

Новый ordinary-attention query $\hat{q}_t$ вычисляет scores относительно past keys, затем softmax-normalized weighted combination values.

Схематически:

$$y_t = \text{softmax} \left(\frac{\hat{q}_t \hat{K}_{\leq t}^\top}{\sqrt{d_h}} \right) V_{\leq t}.$$

Здесь head index для краткости опущен, а scaled dot-product записан явно.

Ключевое свойство read - явная адресация token-level history.

Write

Для нового token вычисляются $\hat{K}_t, V_t$ и добавляются в cache.

Это append-only write.

Forgetting

В plain full attention forgetting нет. Cache растёт вместе с prefix.

Windowing, eviction, quantization или compression будут дополнительными mechanisms, а не свойствами full attention по определению.

Normalization и gates

Основная normalization read - softmax. Полный Transformer block дополнительно содержит собственные normalization, projections, residual paths и FFN.

Training algorithm

При full-sequence processing positions считаются parallel, а causal mask задаёт зависимости. Attention arithmetic по длине sequence имеет quadratic pairwise component.

Decode algorithm

На каждом новом token reuse уже вычисленных past K/V. Новый query читает growing cache.

Именно causal dependency graph делает этот reuse exact: representations уже завершённого prefix не зависят от будущих tokens.

Gated DeltaNet

Теперь тот же checklist для finite recurrent state.

Что хранится

Матрица $S_t \in \mathbb{R}^{d_v \times d_k}$.

Её shape не растёт с длиной prefix.

Read

В базовой ориентации курса: $y_t = S_t q_t$.

Полный block может добавлять normalization и другие transformations, но memory core читается через finite state.

Write и forgetting

Упрощённая scalar-gate форма:

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Здесь работают два разных механизма.

α_t глобально ослабляет старое state.

Term с $\beta_t k_t k_t^\top$ выполняет targeted correction в направлении текущего key.

Term $\beta_t v_t k_t^\top$ записывает новое desired value.

То есть «забывание» здесь не один процесс. Есть global retention и key-directed editing.

Normalization и gates

Уже из core equation видны gates α_t и β_t .

Но полная architecture не заканчивается этой формулой: остаются projections, normalization, feature transformations и block-level design.

Training algorithm

Token-by-token recurrence является корректным algorithm, но для efficient full-sequence training нужны parallel или chunkwise procedures, использующие структуру update.

Сам факт существования recurrence не доказывает, что naive recurrent loop эффективно загрузит GPU.

Decode algorithm

Для autoregressive decode естественен recurrent update: хранится только current finite state, затем для нового token выполняются read и write.

Masked diffusion

Теперь намеренно применим ту же схему к объекту совсем другого типа.

Сразу выясняется важная вещь: masked diffusion **не является ещё одной recurrence для хранения prefix**.

Что является state

Во время generation есть текущая corrupted sequence $X^{(m)} \in \bar{\mathcal{V}}^T$.

Это state самого generative process, а не compressed memory state sequence mixer.

Full bidirectional denoiser при этом обычно не имеет exact static KV-cache для всей изменяющейся sequence между iterations.

Read

На одном NFE denoiser читает текущую corrupted sequence и выдаёт distributions clean tokens для masked positions: $p_\theta(x_t \mid X^{(m)})$.

Read bidirectional внутри доступной denoising region.

Write

Sampler выбирает, какие predictions принять, и изменяет несколько masked positions:

$$X^{(m)} \rightarrow X^{(m+1)}.$$

Это update generative state, а не outer-product write в recurrent memory.

Forgetting

Forward corruption, наоборот, постепенно стирает clean information. Для одной position:

$$q_{\tau}(\tilde{x}_{\tau,t} \mid x_t) = (1 - \rho_{\tau})\mathbf{1}[\tilde{x}_{\tau,t} = x_t] + \rho_{\tau}\mathbf{1}[\tilde{x}_{\tau,t} = \mathbf{MASK}].$$

Generation идёт в обратную сторону и уменьшает corruption.

Поэтому слово forgetting здесь имеет другую semantics, чем $\alpha_t S_{t-1}$ в recurrent mixer.

Normalization и gates

Эти детали принадлежат backbone. Они не определяют masked diffusion как generative construction.

Training algorithm

Сэмплируется clean sequence, noise level τ и corrupted state. Denoiser обучается предсказывать clean categories на masked positions с соответствующим weighting objective.

Не требуется прогонять всю forward trajectory до τ , если marginal corruption можно sample напрямую.

Generation algorithm

Стартуем с heavily corrupted state, часто с all-mask region, и выполняем iterative unmasking.

Количество iterations не следует из одного training loss. Оно задаётся sampler.

Почему это уже защищает от ложной унификации

Посмотрим на три объекта рядом.

Вопрос	Full attention	Gated DeltaNet	Masked diffusion
Persistent representation прошлого	Token-indexed KV-cache	Fixed-shape matrix S_t	Не это является определяющим объектом
Read	Content-addressed attention по cache	$S_t q_t$	Bidirectional denoising текущей corrupted sequence
Write	Append K_t, V_t	Gated targeted state update	Sampler изменяет текущую noisy sequence
Forgetting	В plain variant нет	α_t и delta-like erase	Forward corruption скрывает clean categories
Decode / generation	Один новый token, growing cache	Recurrent state update	Несколько denoising NFE
Главный bottleneck	Cache capacity и traffic	Finite-state capacity, update cost	Repeated sequence processing и NFE

На достаточно абстрактном уровне у всех трёх можно найти «состояние» и «обновление состояния». Но если остановиться на таком уровне, теряется почти всё существенное.

Это и есть пример слишком сильной абстракции.

Фраза

«во всех случаях есть state, который меняется»

истинна, но почти бесполезна.

Она не позволяет вывести ни memory semantics, ни read mechanism, ни cost profile.

Пример

Представим abstract из будущей paper:

We introduce FalconMambaDeltaDiffusion-4, a recurrent architecture with efficient memory and parallel generation.

Название ничего не объясняет.

Начинаем заполнять карточку.

Paper сообщает state: $S_t \in \mathbb{R}^{d_v \times d_k}$.

Дальше находим read: $y_t = S_t q_t$.

Update: $S_t = S_{t-1} A_t + U_t$.

Это пока только generic finite-state form. Нужно раскрыть A_t и U_t .

Допустим, выясняется, что A_t содержит key-dependent rank-one correction. Тогда mechanism действительно оказывается delta-like.

Но paper одновременно использует masked-diffusion objective для output sequence. Это уже другой уровень architecture: sequence mixer относится к внутреннему backbone, а diffusion относится к generative factorization.

Назвать всю систему «delta recurrence» или всю систему «diffusion architecture» недостаточно. Оба утверждения описывают разные axes одной модели.

Где здесь легко ошибиться

Ошибка 1. «У двух architectures похожая state equation, значит это одна architecture».

Нет. State update - только одна design axis.

Ошибка 2. «Если model recurrent, у неё обязательно fixed-size context memory».

Не обязательно. Recurrent outer generation loop может использовать внутри growing KV-cache.

Ошибка 3. «Если model называется diffusion, её backbone не autoregressive Transformer-like».

Generative procedure и sequence mixer - разные уровни решения.

Ошибка 4. «Sparse attention и recurrent state - два варианта одного способа сжать cache».

Нет. Fixed state сжимает representation history. Sparse attention может хранить history целиком, но читать только selected subset.

Ошибка 5. «Достаточно посмотреть main equation».

Main equation очень важна, но после неё всё равно нужно прочитать algorithm section. Один operator может иметь несколько implementations с разным cost vector.

Что теперь понятно

Названия architecture families становятся вторичными.

Рабочее описание новой sequence architecture выглядит так:

state/cache → read → write → forgetting → normalization/gates → training algorithm → decode algorithm.

Для generative model поверх него добавляется:

objective → path → sampler → NFE → processed elements.

Этого языка хватает, чтобы сравнивать full attention, Gated DeltaNet и masked diffusion, не объявляя их искусственно «вариантами одной recurrence».

3. Три смысла слова «эквивалентность»

Зачем эта тема здесь

Слово «эквивалентность» появлялось почти во всех частях курса.

Оно же особенно легко создаёт слишком сильные выводы.

Мы говорили, что causal linear sum эквивалентна recurrence. Говорили, что operator можно считать parallel, recurrent или chunkwise. Обсуждали SSD, delta-family, RWKV-7 и другие state-based systems рядом друг с другом.

Это три разных типа утверждений. Их нужно окончательно разделить.

Интуиция

Есть большая разница между тремя фразами:

1. «Эти две формулы дают один output для любого допустимого input».
2. «Этот operator можно вычислить двумя разными algorithms».
3. «Эти architectures используют похожую идею».

Первые две фразы можно превратить в точную математическую задачу.

Третья - классификация.

Для неё можно привести хорошие аргументы, но обычно нельзя поставить знак равенства между полными models и сказать, что это доказано алгеброй.

Понятия, которые вводятся

Фиксируем три уровня.

Уровень 1 - operator equivalence. Две записи задают одну mathematical function.

Уровень 2 - computational equivalence. Разные algorithms вычисляют один зафиксированный operator.

Уровень 3 - architectural similarity. Разные architectures используют общий design principle.

Первый и второй уровни при конкретных assumptions допускают exact proof.

Третий - taxonomy.

Формальное изложение

Уровень 1. Две формулы задают одну функцию

Возьмём causal linear operator:

$$y_t = \sum_{i=1}^t (q_t^\top k_i) v_i.$$

По associativity: $(q_t^\top k_i) v_i^\top = q_t^\top (v_i k_i^\top)$.

Поэтому:

$$y_t^\top = q_t^\top \sum_{i=1}^t v_i k_i^\top.$$

Определяем

$$S_t = \sum_{i=1}^t v_i k_i^\top.$$

Тогда $y_t^\top = S_t q_t$.

А из определения state: $S_t = S_{t-1} + v_t k_t^\top$.

Никакой approximation здесь не произошло. Мы только перегруппировали конечную сумму.

Это строгая identity двух математических representations.

Уровень 2. Один operator - разные evaluation algorithms

Тот же operator можно вычислить:

- через full pairwise matrix;
- token-by-token recurrently;
- chunkwise.

Chunkwise form разделяет contribution прошлых chunks и текущего chunk:

$$Y_b = Q_b (S_b^{\text{in}})^\top + ((Q_b K_b^\top) \odot M_{c_b}) V_b.$$

State после chunk:

$$S_b^{\text{out}} = S_b^{\text{in}} + K_b^\top V_b.$$

При одинаковых inputs, initial state и causal semantics algorithms вычисляют один operator.

Но их cost vectors различаются.

Literal parallel form может создавать $T \times T$ pairwise structure.

Simple recurrent form хранит state фиксированного shape, но имеет длинную sequence dependence.

Chunkwise form занимает промежуточную точку.

Получается:

equality outputs не означает equality costs.

Уровень 3. Похожий design principle

Теперь сравним Gated DeltaNet, RWKV-7 и KDA.

Все три разумно обсуждать рядом, потому что они работают с finite recurrent memory и развивают идеи управляемого forgetting/editing.

Но из этого не следует:

$$\text{Gated DeltaNet} = \text{RWKV-7} = \text{KDA}.$$

Такого равенства architectures у нас нет.

У них различаются state dynamics, granularity gates, projections и другие block-level choices.

Общий язык полезен как taxonomy:

$$\begin{aligned} \text{additive memory} &\rightarrow \text{forgetting} \rightarrow \text{targeted editing} \\ &\rightarrow \text{finer-grained memory control.} \end{aligned}$$

Эта схема показывает направление design choices. Она не является theorem об equality operators.

Delta rule как online learning

Теперь разберём пример, из-за которого особенно хочется сделать слишком сильное обобщение.

Пусть state S должен по key k_t возвращать value v_t .

Введём local loss

$$\ell_t(S) = \frac{1}{2} \|Sk_t - v_t\|^2.$$

Prediction текущей memory до update: $\hat{v}_t = S_{t-1}k_t$.

Найдём gradient по S : $\nabla_S \ell_t(S) = k_t(Sk_t - v_t)^\top$.

Сделаем один gradient-descent step с learning rate β_t : $S_t = S_{t-1} - \beta_t \nabla_S \ell_t(S_{t-1})$.

Подставляем gradient:

$$S_t = S_{t-1} - \beta_t(S_{t-1}k_t - v_t)k_t^\top.$$

Меняем знак внутри скобок:

$$S_t = S_{t-1} + \beta_t(v_t - S_{t-1}k_t)k_t^\top.$$

Это delta update.

То есть здесь действительно есть точная online-learning interpretation: recurrent memory update совпадает с одним gradient step по локальному quadratic loss.

Это содержательный результат, а не метафора.

Почему отсюда не следует «всё выводится из loss и optimizer»

Соблазн понятен.

Мы взяли memory S , придумали локальный objective, применили optimizer и получили знакомую recurrence. Значит, кажется естественным сказать:

архитектура - это просто loss памяти плюс optimizer.

Проблема в слове «просто».

Наш derivation определил только update переменной S при уже данных k_t и v_t .

Он не ответил:

- откуда берётся k_t ;
- откуда берётся v_t ;
- как строится q_t ;
- какая feature map используется;
- есть ли short convolution перед mixer;
- применяется ли token shift;
- какая normalization используется;
- какие дополнительные gates стоят вокруг update;
- как параметризована discretization;
- как устроены residual paths;
- каким FFN окружён mixer;
- как смешиваются разные layers;
- есть ли attention layers;
- как recurrence parallelizes при training.

Более того, local memory loss $\ell_t(S)$ не нужно путать с внешним training objective параметров всей neural network.

Внутри forward pass можно интерпретировать state update как optimization step по ℓ_t . Сама model при этом может обучаться обычным language-model objective, diffusion objective или другим outer loss.

Это два уровня optimization.

Именно здесь слишком сильная унификация ломается.

Пример

Возьмём две hypothetical models A и B.

Обе используют один и тот же update:

$$S_t = S_{t-1} + \beta_t(v_t - S_{t-1}k_t)k_t^\top.$$

Но в A k_t и v_t получаются обычными linear projections текущего hidden state.

В B перед projections стоит causal short convolution, используется другая normalization, vector gate, а каждый четвёртый layer заменён full attention.

Локальный memory update совпадает.

Полные architectures - нет.

У них могут различаться:

- outputs при одинаковом raw input;
- training dynamics;
- context capacity;
- $M_{\text{persistent}}$;
- B_{HBM} ;
- kernel efficiency;
- inductive bias.

Поэтому equality одной update formula недостаточно для equality models.

Где здесь легко ошибиться

Ошибка 1. «SSD доказал, что SSM и Transformer одинаковы».

Нет. Доказанные связи относятся к конкретным structured operators и representations.

Ошибка 2. «Если recurrence выводится из optimization problem, вся architecture тоже выводится из неё».

Нет. Выведен один state update при фиксированных представлениях.

Ошибка 3. «Одинаковая формула update означает одинаковые outputs всей сети».

Нет. Inputs этой формулы и окружающие blocks могут различаться.

Ошибка 4. «Architectural similarity можно доказать тем же способом, что equality двух sums».

Нет. Similarity - выбранная taxonomy.

Ошибка 5. «Local loss памяти - это loss, которым обучают model weights».

Не обязательно. Inner online update и outer training objective нужно разделять.

Что теперь понятно

Слово «эквивалентность» без уточнения уровня лучше вообще не использовать.

Нужно сказать одно из трёх:

1. Эти формулы определяют одну mathematical function.
2. Эти algorithms точно вычисляют один operator.
3. Эти architectures используют похожий design principle.

Первые два утверждения могут иметь algebraic proof.

Третье полезно для карты design space, но остаётся классификацией.

Online-learning interpretation delta rule относится к первому типу локального математического утверждения об update. Она не превращает архитектуру целиком в «loss + optimizer».

4. Как читать новую статью

Зачем эта тема здесь

Когда выходит новая architecture paper, естественный порядок чтения часто получается обратным нужному.

Сначала видим headline: 2.4x faster .

Потом benchmark table.

Потом название нового block.

И только в конце доходим до equation и выясняем, что именно было изменено.

Для технического разбора порядок полезнее перевернуть.

Интуиция

Paper обычно отвечает на четыре разных вопроса:

1. Что вычисляется?
2. Как это вычисляется?
3. Почему авторы ожидают efficiency на конкретном hardware?
4. Что получилось в experiment?

Если смешать уровни, empirical speedup начинает выглядеть как theorem, а красивое asymptotic выражение - как гарантия wall-clock latency.

Поэтому рабочий порядок такой:

mathematical object → algorithm → hardware claim → benchmark.

Benchmark идёт последним не потому, что он неважен. Наоборот, именно так становится понятно, что benchmark проверяет.

Понятия, которые вводятся

Новых математических понятий нет. Вводится рабочая карточка разбора paper.

Карточка новой архитектуры

1. Какую задачу решает layer или generative mechanism?
2. Каково основное equation?
3. Что хранится как state или cache?
4. Как выполняется read?
5. Как выполняется write/update?
6. Что забывается?
7. Есть ли parallel, recurrent или chunkwise representation?
8. Какова training complexity?
9. Какова decode/generation complexity?
10. Какие hardware assumptions используются?
11. Какая precision предполагается?
12. Какой objective используется?
13. Какой probability/noise path?
14. Какой sampler?
15. Сколько N_{FE} ?
16. Сколько элементов обрабатывается на один NFE?
17. С каким baseline сравнивают?
18. Что доказано exact identity или theorem?
19. Что только измерено авторами?
20. Какие failure modes или bottlenecks остаются?
21. Как попытаться опровергнуть своё понимание: проверить shapes и orientation, causal dependencies, limiting cases, independent reference, outputs, gradients, sensitivity к precision и hardware profile?

Не каждая paper заполнит все клетки. Например, у обычного sequence mixer нет собственного generative sampler. Тогда так и пишем: «не применимо».

Формальное изложение

Почему сначала mathematical object

Предположим, авторы называют layer HyperLinearMemory .

Это имя не сообщает, является ли operator:

- dense attention;
- sparse attention;
- fixed-state recurrence;
- convolution;
- комбинацией нескольких branches.

Equation сообщает.

Пока main operator не выписан, слова `linear`, `memory` и `attention` в названии классифицируют paper хуже самой формулы.

Почему потом **algorithm**

Даже точное equation не даёт hardware cost.

На занятии 1 один operator имел parallel, recurrent и chunkwise forms.

Поэтому после вопроса «что вычисляется?» сразу должен идти вопрос:

как именно авторы это вычисляют в нужном режиме?

Training и decode могут использовать разные algorithms.

Это особенно характерно для state-based models: recurrent representation удобна для decode, а training требует parallelization или chunking.

Почему **hardware claim** только третьим

Из algorithm уже можно увидеть:

- shapes matrix multiplications;
- data reuse;
- sequential dependencies;
- объём reads/writes;
- возможность kernel fusion.

Только после этого фраза «hardware efficient» приобретает технический смысл.

Даже тогда это ещё не universal theorem.

Например, throughput зависит как минимум от:

- GPU architecture;
- memory bandwidth;
- dtype;
- batch size;
- sequence length;
- state dimensions;
- number of heads;
- kernel implementation;
- fusion;
- concurrency;
- framework overhead;
- baseline implementation.

Почему benchmark последний

Benchmark отвечает на empirical question:

что произошло в условиях experiment?

Это очень важный вопрос, но он уже не математического claim.

Пусть paper сообщает шестикратное ускорение.

Корректное чтение:

В реализации авторов, на указанном hardware, при указанном batch, dtype, context length и baseline был измерен throughput gain 6x.

Некорректное чтение:

Эта recurrence в шесть раз быстрее Transformer.

Вторая фраза превратила measurement в invariant architecture.

Доказано и измерено

Полезно буквально делать две колонки.

Доказано / следует из определения	Измерено
Causal sum равна выведенной recurrence	Throughput конкретного kernel
Fixed-shape state не растёт с T при фиксированных dimensions	Реальная peak GPU memory
Exact causal dependency разрешает reuse past KV	Tokens/s конкретной implementation
Chunkwise decomposition вычисляет тот же operator	Benchmark accuracy
Block-causal dependency делает completed prefix invariant	Ускорение block diffusion в implementation авторов
Конкретная asymptotic оценка при заданном algorithm	Latency на H100, H800 или другом accelerator

Даже asymptotic analysis требует assumptions, но по природе это всё равно другое утверждение, чем measurement.

Held-out case

Рассмотрим условную Method X, которой в курсе не было. Paper задаёт layer:

$$S_t = D_t S_{t-1} + u_t v_t^\top, \quad y_t = g_t^\top S_t.$$

Здесь $S_t \in \mathbb{R}^{d_s \times d_v}$, $D_t = \text{diag}(d_t)$, $d_t \in (0, 1)^{d_s}$, а $u_t, g_t \in \mathbb{R}^{d_s}$ и $v_t \in \mathbb{R}^{d_v}$.

Обратите внимание: состояние здесь транспонировано относительно нашей записи - переход действует слева, чтение идёт вектором-строкой $g_t^\top S_t$. Так бывает постоянно: разные статьи держат разную ориентацию, и первый шаг разбора - понять, какая сущность стоит на какой оси, а не сверять буквы. В нашей записи тот же layer выглядел бы как $S_t = S_{t-1} D_t + v_t u_t^\top$, $y_t = S_t g_t$. Для full-sequence training авторы используют parallel scan по diagonal transitions; autoregressive decode выполняется recurrently. Полная generative model использует autoregressive factorization.

Разбор по карточке:

1. **State equation:** fixed-shape matrix S_t .
2. **Read:** $g_t^\top S_t$.
3. **Write:** rank-one term $u_t v_t^\top$.
4. **Forgetting:** channel-wise diagonal transition D_t .
5. **Operator и evaluation algorithm:** recurrence и parallel scan — разные уровни описания.

6. **Ось A:** finite context state с channel-wise forgetting.
7. **Ось B:** autoregressive; она не следует из recurrence самого layer.
8. **Cost claim:** fixed state говорит о scaling persistent context state, но сам по себе не доказывает меньшую latency.
9. **Проверка:** special case $D_t = I$ должен убрать forgetting; causal dependence нужно проверить отдельно; recurrent и scan implementations нужно сравнить с independent naive reference.

Знакомый worked example — Gated DeltaNet

Заполним карточку для Gated DeltaNet на уровне материала курса.

1. Задача

Организовать finite recurrent memory, в которой можно отдельно:

- глобально ослаблять старое содержимое;
- адресно исправлять association по текущему key.

2. Основное equation

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

3. State

$$S_t \in \mathbb{R}^{d_v \times d_k}.$$

Shape не растёт с длиной prefix.

4. Read

В базовой notation: $y_t = S_t q_t$.

5. Write

Targeted delta-like update с strength β_t .

6. Forgetting

α_t выполняет global/data-dependent forgetting.

Rank-one correction, связанная с k_t , редактирует memory в конкретном направлении.

7. Parallel / recurrent / chunkwise

Recurrent form естественна из equation.

Для practical training нужен structured parallel/chunkwise algorithm. Сам naive token loop не является достаточным аргументом о training speed.

8. Training complexity

По длине sequence state update допускает algorithms без dense full-attention pairwise matrix, но actual training cost определяется выбранной parallelization, dimensions и kernels.

Одного слова «linear» недостаточно для latency claim.

9. Decode complexity

Хранится fixed-shape recurrent state; на каждом token выполняются read и update.

Persistent state по T не растёт.

10. Hardware assumptions

Преимущество зависит от наличия эффективных kernels для structured state update, подходящих tensor shapes, dtype и batch regime.

11. Precision assumptions

Нужно смотреть implementation и experiments paper. Из одной recurrence precision regime не следует.

12. Generative objective

Сам Gated DeltaNet layer его не задаёт.

Если backbone обучается как autoregressive LM, objective приходит от полной generative model.

13. Sampler

Также не свойство layer. Для обычной AR LM sampler работает поверх autoregressive token distributions.

14. Baseline

Нужно читать experiment section и проверять, равны ли model scale, training budget, context и implementation quality.

15. Что здесь доказано

Из local loss

$$\ell_t(S) = \frac{1}{2} \|Sk_t - v_t\|^2$$

можно точно вывести delta update.

Можно также алгебраически проверить special cases update при конкретных α_t, β_t .

16. Что измерено

Quality модели, throughput, memory consumption и reported speedups являются empirical results конкретной paper и implementation.

17. Failure mode

Finite-state bottleneck никуда не исчезает. Более умное управление state улучшает utilization, но state всё равно имеет конечную размерность.

Карточка уже даёт содержательное описание architecture, даже если удалить из неё слова Gated DeltaNet .

Где здесь легко ошибиться

Ошибка 1. Сначала читать benchmark table, а equation потом.

Тогда непонятно, какой intervention benchmark вообще проверяет.

Ошибка 2. «Authors report 6x speedup» превращать в «architecture is 6x faster».

Второе утверждение намного сильнее первого.

Ошибка 3. Сравнивать methods по разным regimes.

Prefill throughput одной model нельзя напрямую сравнивать с token-by-token decode latency другой.

Ошибка 4. Считать theoretical FLOPs достаточным hardware model.

Memory traffic и sequential structure могут изменить bottleneck.

Ошибка 5. Читать отсутствие failure-mode section как отсутствие failure modes.

Нужно самому искать, какой ресурс или structural bottleneck method не устранил.

Ошибка 6. Не проверять baseline.

Слабый или неоптимизированный baseline может дать большой relative speedup, который плохо переносится на production implementation.

Что теперь понятно

Новая paper читается не как рассказ авторов о своём названии, а как набор проверяемых claims.

Сначала:

operator.

Потом:

algorithm.

Потом:

hardware assumptions.

И только затем:

measurements.

Первое действие при любом громком результате - разделить, что доказано, а что измерено.

5. Границы курса

Зачем эта тема здесь

Система координат полезна только тогда, когда понятно, чего в ней пока нет.

Семь занятий не покрывают весь design space generative architectures. Более того, несколько важных современных направлений естественно продолжают линии курса, но требуют отдельных занятий.

Здесь их нужно не «успеть пройти», а правильно разместить на карте.

Интуиция

У каждого пропущенного направления можно спросить:

какую координату или какой элемент decomposition оно в основном меняет?

Тогда новые темы не выглядят отдельными мирами.

TTT продолжает вопрос update finite memory.

Cache quantization и eviction продолжают вопрос representation explicit history.

MoE в основном меняет cost обработки weights, а не memory semantics context.

Speculative decoding меняет последовательность expensive autoregressive evaluations, не превращая causal LM в diffusion.

Это уже достаточно, чтобы понимать, где искать продолжение.

Понятия, которые вводятся

Новых обязательных понятий нет. Четыре направления остаются outside core:

- TTT и learned-optimizer memories;
- quantization, compression и eviction KV-cache;

- Mixture of Experts;
- speculative decoding и multi-token prediction.

Формальное изложение

TTT и learned-optimizer memories

Delta rule показал простейший случай:

memory update = optimization step по **local objective**.

TTT расширяет именно эту ось: fast state может быть более общей learnable model, которая обновляется во время обработки sequence с помощью внутренней learning procedure.

То есть естественный следующий вопрос после delta rule:

что произойдёт, если memory itself станет маленькой обучаемой model, а update - более общим optimizer?

Это содержательное продолжение курса, но уже новая тема. Нужно обсуждать inner objective, optimizer, parameterization memory, stability и efficient parallelization.

Quantization, compression и eviction cache

Full attention не обязательно хранит каждый cached element в исходной precision и бесконечно долго.

Есть как минимум три независимых идеи.

Quantization уменьшает bytes на cached representation.

Если число entries всё равно растёт с T , asymptotic growth по длине context не исчезает, но coefficient уменьшается.

Compression меняет representation cached history.

Eviction решает, какие entries больше не хранить.

В нашем языке это варианты изменения $M_{\text{persistent}}$, B_{HBM} и иногда semantics read.

Они не превращаются автоматически в fixed recurrent state.

Mixture of Experts

MoE отвечает в первую очередь на другой вопрос:

обязаны ли все parameters network участвовать в обработке каждого token?

Total weights и active weights становятся разными величинами.

Это ещё один хороший контрпример против scalar «размера модели».

Model может иметь большой M_{weights} , но активировать для token только subset experts и тем самым менять F .

При этом появляются routing, communication и load-balancing costs.

MoE не отвечает само по себе на вопрос, хранится ли context в KV-cache или recurrent state. Эти axes можно комбинировать.

Speculative decoding

Обычная autoregression имеет sequential dependency:

$$x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T.$$

Speculative decoding пытается уменьшить число дорогих последовательных evaluations target model, предлагая несколько будущих tokens более дешёвым mechanism и проверяя их группой.

Главный вопрос здесь не memory representation, а:

сколько дорогих target-model stages требуется на один accepted token?

Это воздействует прежде всего на D_{seq} , число expensive evaluations и hardware utilization.

Но выигрыш зависит от acceptance rate, draft cost, verification algorithm и batch regime.

Multi-token prediction

Multi-token prediction добавляет способность предсказывать несколько будущих positions вместо одного next token.

Это может быть полезно само по себе как training signal и может использоваться в inference mechanisms, которые пытаются принимать несколько tokens за один дорогой stage.

Но фраза «МТР уменьшает число проходов» требует указать конкретный decode algorithm. Наличие multi-token heads ещё не гарантирует, что production sampler действительно принимает несколько tokens после каждого target evaluation.

Так что здесь сохраняется знакомое правило:

сначала algorithm, потом claim о числе проходов.

Пример

Представим две новые papers.

Paper A сообщает 4-bit KV cache .

Paper B сообщает 4 tokens per target-model verification .

Обе рекламируются как inference efficiency.

Но они атакуют разные координаты.

Paper A в первую очередь меняет bytes persistent cache и traffic.

Paper B пытается уменьшить sequential cost autoregressive decode.

Сказать «обе делают inference в четыре раза дешевле» нельзя.

Число 4 стоит в двух совершенно разных физических величинах.

Где здесь легко ошибиться

Ошибка 1. «Квантизация cache решила рост с context length».

Если сохраняются entries всех tokens, количество entries всё равно растёт с T .

Ошибка 2. «MoE model с 400B parameters вычисляет 400B parameters на каждом token».

Не обязательно. Нужно различать total и activated parameters.

Ошибка 3. «Speculative decoding убирает autoregressive factorization».

Нет. Final distribution target model остаётся autoregressive; меняется evaluation strategy.

Ошибка 4. «ТТТ доказывает, что все sequence models - optimizers».

Нет. Это ещё один design family memory updates, а не theorem о всех architectures.

Ошибка 5. «МТР означает несколько final tokens за любой один forward».

Это зависит от конкретной inference procedure.

Что теперь понятно

Границы курса не совпадают с границами системы координат.

Новые mechanisms можно продолжать раскладывать по тем же вопросам:

- какое state или cache;
- какой read;
- какой write;
- какой forgetting;
- какой algorithm;
- какая координата cost vector меняется;
- какой objective и sampler;
- что доказано;
- что измерено.

Именно поэтому framework должен пережить конкретные названия моделей.

Главная схема занятия

Для незнакомой architecture полезен следующий порядок.

Шаг 1. Удалить название

Мысленно заменить Mamba-4 , HyperAttention или TurboDiffusion на Method X .

Если после этого объяснение развалилось, скорее всего мы знали бренд, а не механизм.

Шаг 2. Найти mathematical object

Выписать main equation и shapes.

Не пересказывать prose paper, пока equation не понятна.

Шаг 3. Выписать memory semantics

Что хранится?

Как читается?

Как записывается?

Что забывается?

Шаг 4. Отделить operator от algorithm

Есть ли recurrent, parallel или chunkwise form?

Что реально запускается при training?

Что реально запускается при decode?

Шаг 5. Заполнить cost vector

Не обязательно сразу получать точные числа. Сначала достаточно понять scaling и bottlenecks каждой координаты.

Шаг 6. Если это generative model - разобрать generative construction

Какой objective?

Какой path?

Какой sampler?

Сколько NFE?

Сколько элементов обрабатывает один NFE?

Шаг 7. Разделить proof и measurement

Что является identity или theorem?

Что получено benchmark?

На каком hardware и в какой precision?

Шаг 8. Проверить baseline

С чем именно сравнивают?

Совпадают ли model scale, context, batch, implementation quality и training budget?

Шаг 9. Найти оставшийся bottleneck

Если paper уменьшила cache, что стало с compute?

Если уменьшила FLOPs, что стало с traffic?

Если уменьшила NFE, насколько тяжёл один NFE?

Если ввела fixed state, что произошло с memory capacity?

Шаг 10. Проверить и попытаться опровергнуть

Проверить shapes и orientation.

Проверить causal dependencies.

Проверить limiting и special cases.

Написать независимую naive reference implementation.

Сравнить outputs с reference.

Если речь об обучении, сравнить gradients.

Проверить sensitivity к precision.

Отдельно profile target hardware и regime.

Совпадение двух optimized implementations между собой не доказывает correctness: нужен независимый эталон.

После этого название architecture можно вернуть. Теперь оно является короткой ссылкой на разобранный mechanism, а не заменой объяснения.

Сводная таблица курса

В таблице ниже complexity дана схематически. Конкретные constants, число heads, layers, dtype и implementation могут существенно менять practical cost.

Клас	Context/backbone state или cache	Generate/sampled state	Read	Write / update	Что забывается	Training и decoding	Ключевая координата cost vector
Full causal attention	Token-indexed KV-cache	Generated prefix при autoregressive generation	Softmax attention по past K/V	Append нового K/V	В plain variant ничего	Full sequence имеет pairwise $O(T^2)$ component; decode читает growing cache	$M_{\text{persistent}}$, B_{HBM} , F
GQA / MQA	Token-indexed KV-cache с меньшим числом distinct KV-heads	Generated prefix при autoregressive generation	Query-heads читают shared K/V heads	Append shared K/V	Обычно ничего по определению	Уменьшается head-count coefficient KV-cache без обязательного сжатия каждой token representation	$M_{\text{persistent}}$, B_{HBM}
Compressed KV / MLA-like cache	Token-indexed compressed или latent representation прошлого	Generated prefix при autoregressive generation	Attention использует сохранённое compressed representation и architecture-	Append compressed token representation	Обычно ничего по определению	Уменьшаются bytes или latent dimensions на cached token; механизм отличается	$M_{\text{persistent}}$, B_{HBM} , F

Класс	Context/backbone state или cache	Generate/sample state	Read	Write / update	Что забывается	Training и decoding	Ключевая координата cost vector
			specific reconstruction/read			от sharing KV-heads	
Additive linear state	$S_t \in \mathbb{R}^{d_v \times d_k}$	Generate d prefix при autoregressive LM; иначе задаётся full generative model	$S_t q_t$	$S_t = S_{t-1} + v_t k_t^T$	Ничего	Recurrent decode имеет fixed state; training требует parallel/chunkwise implementation для efficiency	$M_{\text{persistent}}$, D_{seq}
Decayed state / retention	Fixed-shape state	Generate d prefix при autoregressive LM; иначе задаётся full generative model	Read из decayed state	Новый outer product плюс retention старого	Старые записи глобально затухают	Decode recurrent; structured parallel/chunkwise forms возможны для конкретных operators	$M_{\text{persistent}}$, D_{seq} , F
Delta rule	Fixed matrix state	Generate d prefix при autoregressive LM; иначе	$S_t q_t$	Записывается prediction error	Read по current key корректируется; неортого	Decode recurrent; efficient training требует	State capacity, F , D_{seq}

Класс	Context/backbone state или cache	Generative/sample state	Read	Write / update	Что забывается	Training и decoding	Ключевая координата cost vector
		задаётся full generative model			нальные directions могут изменить ся	structured algorithm	
Gate d Delta Net	Fixed matrix state	Generate d prefix при autoregressive LM; иначе задаётся full generative model	Read из finite state	Delta-like targeted correction	Global gate α_t плюс key-directed correction	Fixed-state decode; training cost зависит от parallel/chunkwise kernel	$M_{\text{persistent}}$, F , D_{seq}
SSD / Mamba-2	Structured finite state	Задаётся полной generative model; в AR LM — generated prefix	Structured output map из state	Structured state transition	Определяется parameterized transition	Operator допускает state-space и structured matrix views; practical training использует hardware-aware algorithms	F , D_{seq} , B_{HBM}
RWK V-7	Finite recurrent state	Задаётся полной generative	Architecture-specific	Generalized delta-	Vector-valued control	Decode recurrent; training	State utilization,

Класс	Context/backbone state или cache	Generative/sample state	Read	Write / update	Что забывается	Training и decoding	Ключевая координата cost vector
		е model; в AR LM — generated prefix	recurrent read	like update		использует специализированную parallelization	F , D_{seq}
KDA	Finite recurrent state	Задаётся полной generative model; в AR LM — generated prefix	Delta-family read	Delta-family editing c finer-grained gating	Более fine-grained / channel-wise control	Fixed-state decode; training зависит от specialized kernels	State utilization, F , B_{HBM}
Hybrid state + attention	И fixed states, и token-indexed KV-cache	Задаётся полной generative model; в AR LM — generated prefix	Разные layers или branches читают разную память	Оба типа update	Зависит от components	Growing cache остаётся там, где остаётся attention; state layers меняют coefficient и compute profile	Весь профиль, особенно $M_{persistent}$
Sparse attention	Обычно большой token-indexed KV pool	Задаётся полной generative model; в AR LM	Selected blocks или positions	Обычно append в underlying pool	Не обязательно забывается	Storage может расти как раньше, но main read и arithmetic	B_{HBM} , F , иногда $M_{persistent}$

Клас	Context/backbone state или cache	Generate/sample state	Read	Write / update	Что забывается	Training и decoding	Ключевая координата cost vector
		— generated prefix			ся вообще	сокращаются	
Flow matching	Зависит от backbone architecture	Текущий sample x_τ	Neural vector field $v_\theta(x_\tau, \tau)$	Numerical solver изменяет x_τ	He memory forgetting, а движение по probability path	Training samples path points; generation требует solver NFE	N_{FE} , F , D_{seq}
Gaussian diffusion / score	Зависит от denoising backbone	Noisy sample x_τ	Score или эквивалентная denoising parameterization	Reverse sampler обновляет sample	Forward process уничтожает signal noise-ом	Training обычно samples noise level; generation iterative	N_{FE} , per-NFE F , D_{seq}
Masked diffusion	Зависит от denoiser; exact static full-sequence KV-cache между iterations обычно нет	Current corrupted token sequence	Bidirectional probability denoiser	Sampler unmask/ revises positions	Forward process переводит tokens в mask state	Full-sequence generation может заново process все positions на каждом NFE	N_{FE} , $N_{\text{processed}}$, F

Класс	Context/backbone state или cache	Generative/sample state	Read	Write / update	Что забывается	Training и decoding	Ключевая координата cost vector
Block diffusion	KV-cache completed immutable blocks	Current diffusion block / partial sample	Bidirectional внутри current block, causal read completed prefix	Denoising current block, затем block становится immutable prefix	Noise уменьшается внутри current block	Между blocks появляется safe KV reuse; внутри block остаётся iterative denoising	N_{FE} , $N_{processed}$, $M_{persistent}$, B_{HBM}

У этой таблицы нет строки «лучшая architecture».

Это не пропуск.

По разным координатам cost vector и для разных задач порядок будет разным.

Вопросы для самопроверки

1. Почему нельзя сравнить две architectures одним числом FLOPs?
2. Две models занимают одинаковые 12 GiB persistent context memory. Означает ли это одинаковый HBM traffic?
3. Две diffusion models имеют $N_{FE} = 8$. Одинакова ли их стоимость generation?
4. Что является state full causal attention при autoregressive decode?
5. Что является state Gated DeltaNet?
6. Что такое state masked diffusion?
7. Почему похожая recurrence не доказывает equality architectures?
8. В каком смысле causal linear sum и recurrent state эквивалентны?
9. В каком смысле parallel и chunkwise forms эквивалентны?
10. Являются ли Gated DeltaNet и RWKV-7 «эквивалентными»?
11. Как delta rule связан с online learning?
12. Почему из предыдущего ответа не следует, что вся architecture определяется loss и optimizer?
13. Нужно ли считать local memory loss $\ell_t(S)$ training objective всей network?
14. Paper доказала $O(T)$ arithmetic вместо $O(T^2)$. Доказала ли она

меньшую wall-clock latency?

15. Paper сообщает $5\times$ throughput gain. Что именно можно утверждать?

16. Почему exact KV-cache корректен для causal Transformer?

17. Почему full bidirectional masked diffusion обычно не может reuse тот же static cache между denoising iterations?

18. Почему block diffusion снова разрешает cache?

19. Что первым нужно проверить у claim «новая model использует меньше памяти»?

20. Что первым нужно сделать с условной Mamba-4 2027 года?

Ответы к вопросам для самопроверки

1. Почему нельзя сравнить две architectures одним числом FLOPs?

Потому что FLOPs не определяют memory traffic, sequential depth, workspace, kernel utilization и concurrency. Меньший F может сочетаться с худшей latency.

2. Две models занимают одинаковые 12 GiB persistent context memory. Означает ли это одинаковый HBM traffic?

Нет. Одна может читать весь stored context на каждом query, другая - небольшой selected subset.

3. Две diffusion models имеют $N_{FE} = 8$. Одинакова ли их стоимость generation?

Нет. Нужно узнать хотя бы число processed elements на NFE, backbone cost, attention structure, caching и precision.

4. Что является state full causal attention при autoregressive decode?

Token-indexed KV-cache. В отличие от fixed recurrent state, число cached entries растёт с prefix.

5. Что является state Gated DeltaNet?

Finite matrix state S_t , shape которого при фиксированных dimensions не зависит от длины prefix.

6. Что такое state masked diffusion?

Текущая corrupted sequence является state generative process. Это не тот же тип memory state, что S_t у recurrent sequence mixer.

7. Почему похожая recurrence не доказывает equality architectures?

Потому что architecture дополнительно определяет inputs recurrence, projections, feature maps, normalization, gates, convolutions, residual structure, layer mixing и другие mechanisms.

8. В каком смысле causal linear sum и recurrent state эквивалентны?

Они задают одну mathematical function. Равенство выводится алгебраически через regrouping outer products.

9. В каком смысле parallel и chunkwise forms эквивалентны?

Это разные evaluation algorithms одного operator. При одинаковой causal semantics они дают тот же mathematical output.

10. Являются ли Gated DeltaNet и RWKV-7 «эквивалентными»?

Не в смысле equality полных architectures. Их можно объединять на уровне design taxonomy как finite-state mechanisms с delta-like memory editing.

11. Как delta rule связан с online learning?

Для

$$\ell_t(S) = \frac{1}{2} \|Sk_t - v_t\|^2$$

один gradient step по S даёт delta update.

12. Почему из предыдущего ответа не следует, что вся architecture определяется loss и optimizer?

Потому что derivation определяет только update memory variable S . Он не определяет feature maps, projections, normalization, gates, convolutions, discretization и layer mixing.

13. Нужно ли считать local memory loss $\ell_t(S)$ training objective всей network?

Нет. Это inner objective для интерпретации state update. Outer model parameters могут обучаться другим objective.

14. Paper доказала $O(T)$ arithmetic вместо $O(T^2)$. Доказала ли она меньшую wall-clock latency?

Нет. Для latency нужны ещё traffic, parallelism, kernels, hardware и regime.

15. Paper сообщает $5\times$ throughput gain. Что именно можно утверждать?

Что authors measured такой gain в указанной experimental configuration. Перенос на другой hardware и implementation является новым experiment.

16. Почему exact KV-cache корректен для causal Transformer?

Потому что representation старой position не зависит от будущих positions. После append нового token старые K/V остаются valid.

17. Почему full bidirectional masked diffusion обычно не может reuse тот же static cache между denoising iterations?

Потому что изменившаяся position может изменить representations других positions через bidirectional dependencies. Старые higher-layer K/V становятся stale.

18. Почему block diffusion снова разрешает cache?

Completed blocks не зависят от current future block. Пока current block меняется, representations completed prefix остаются invariant.

19. Что первым нужно проверить у claim «новая model использует меньше памяти»?

Какой именно memory component: weights, persistent context, workspace или peak runtime allocation.

20. Что первым нужно сделать с условной Mamba-4 2027 года?

Не искать, на какую старую Mamba она «похожа», а выписать main equation, state/cache, read, write, forgetting, training/decode algorithms и cost profile.

Что читать дальше

После этого курса есть несколько естественных направлений.

TTT и learned-optimizer memories. Это прямое продолжение online-learning view delta rule. Имеет смысл смотреть работы, где recurrent state является более общей fast model, обновляемой внутренним optimization process. При чтении сразу применять тот же checklist: memory parameterization, inner objective, optimizer, parallelization и decode state.

KV-cache engineering. Quantization, compression и eviction показывают, что выбор не сводится к «full cache или fixed state». Полезно отдельно изучить, что меняется в bytes per token, что в number of stored tokens, а что в read traffic.

Mixture of Experts. Это отдельная axis cost vector: total parameter storage, activated parameters, routing и communication. MoE особенно полезно рассматривать рядом с context architecture, чтобы не путать sparsity weights и sparsity attention.

Speculative decoding. Здесь основная тема - как уменьшить число expensive sequential target-model evaluations без изменения final autoregressive model distribution.

Multi-token prediction. Полезно читать вместе со speculative decoding и каждый раз проверять, превращается ли multi-token prediction в реальное принятие нескольких tokens при inference или остаётся training mechanism.

Для уже разобранных линий курса полезно вернуться к исходным работам про Structured State Space Duality / Mamba-2, Gated DeltaNet, RWKV-7, Kimi Linear / KDA, MDLM, Discrete Flow Matching и Block Diffusion, но читать их теперь не подряд от abstract до appendix, а по карточке этого занятия. Ссылки на все эти работы - в разделах «Что читать дальше» конспектов занятий 3-6; здесь они не повторяются.

Для frontier-работ и новых версий после материалов курса bibliographic status, version и reported numbers нужно **проверять по актуальному первоисточнику**.

Последняя рабочая формула курса поэтому не equation конкретной architecture, а процедура:

state/cache → read → write → forgetting
→ algorithm → cost vector
→ objective/path/sampler → proof vs measurement.

Если эта цепочка заполнена, новую архитектуру уже можно обсуждать содержательно, даже если её название появилось после окончания курса.